Introduction to ReactJS :

=> What is ReactJS? It's a Javascript library developed by Meta used for building fast and interactive UIs using reusable components.

=> Why useful?
- Reusable code
- Faster websites
- Easy to manage large projects

Create-react-app :

=> command to create react app:

```
npx create-react-app my-app
```

npx → Run packages directly
create-react-app → tool to create a React project
my-app → Project name

=> Project structure:

```
v my-app                      <- contains all installed packages
  > node_modules
  v Public                    <- contains static file
      index.html
  v src                       <- main working folder
      App.js                  <- Main react component.
      index.js                <- Entry point of React App
```

=> To run react app:

```
npm start
```

Opens at: http://localhost:3000

Understanding JSX :

=> What is JSX? It is a syntax used in React to write UI code in an HTML-like way inside JS.
It allows us to write:
- HTML like code
- Inside JS

=> JSX is Javascript syntax that looks similar to HTML.

=> Browser don't understand JSX directly. React converts JSX into JS. This conversion is done by Babel.

=> Ex: to identify JSX. App.js:

```
function App(){
    return (
        <div>
        Aditya
        </div>
    );
}
export default App;
```

function App(){ ← component
 Aditya → JSX
O/P: Aditya

=> JSX Rules:
Rule1: Return only one parent element.
Rule2: Every tag must be closed.
Rule3: use className instead of class
Rule4: JS inside {} JSX allows JS using curly braces.

=> what is a Fragment? A fragment (<></>) is used to group multiple JSX elements without adding an extra tag to the browser.

=> Ex: using one parent tag instead of fragments. App.js:

O/P: Aditya Utsav

```
function App(){
  return (
    <div>
    <h1> Aditya </h1>
    <div> utsav </div>
    </div>
  );
}
export default App;
```

this tag actually returned one parent element.

=> Ex: using fragments for above example ← App.js

O/P: Aditya utsav

```
function App(){
  return (
    <>
    <h1> Aditya</h1>
    <div>utsav </div>
    </>
  );
}
export default App;
```

remember to close

=> Ex: what if neither fragments nor parent element exist? Error!

App.js:

O/P: Error

```
function App(){
  return (
    <h1> A </h1>
    <div> B </div>
  );
}
export default App;
```

Setup + Adding Bootstrap to React:

=> What is Single Page Application (SPA)? It means the website that has only one HTML page, and content changes dynamically without full page reload. e.g., Gmail, Netflix.

=> How SPA works?

1. Browser loads one HTML file.
2. React loads JS
3. React controls the entire UI
4. When you click something:
   - Only part of the page updates
   - Page doesn't reload.

=> What is MultiPage Application? (MPA)

Each page:
- Sends new request to server
- Loads full HTML again.

=> What is node_modules folder? ↳ big in size

It contains:
- React library
- Babel tools
- other dependencies etc.

=> Command to install node_modules folder if got deleted:

```
npm install
```

=> To use bootstrap in react use the CDN method.

understanding Props and Proptypes in React:

=> What are exports? we uses export to share code from one file to another.

=> Types of exports:

1. export default
2. named export

Feature	Named Export	Default Export
How many per file	Many	only 1
Syntax	{a}	anyName
Name flexibility	fixed	flexible
use	Utilities, multiple value	main Comp./value

=> Ex: Named export

M1.mjs:

```
export const a = "Aditya";
export const b = "Utsav";
```

M2.mjs:

```
import {a,b} from './M1.mjs';
console.log(a);
console.log(b);
```

O/P: Aditya Utsav

=> Ex: export default

O/P: Utsav

M1.mjs:

```
const a = "Aditya";
const b = "Utsav";
export default b;
```

M2.mjs:

```
import any_name from './M1.mjs';
console.log(any_name);
```

=> What are Props? Props = Properties. Props are used to pass data from one component to another in React.

=> Ex: without props. App.js:

```
function Adi(){
  return (
    <> <h1> Aditya </h1> </>
  )
}
```

```
function App(){
  return (
    <> <Adi/>
      <Adi/>
    </>
  )
}
export default App;
```

O/P: Aditya Aditya

=> Ex: with props. App.js: (recieving data)

```
function Greet(props){
  return (
    <>
    <h1> Hello {props.name} </h1>
    </>
  )
}
```

(using data)

O/P: Hello A Hello B

```
function App(){
  return (
    <>
    <Greet name="A"/>
    <Greet name="B"/> </>
  )
}
export default App;
```

(Prop)

=> we can pass integer as:

Number = {20}

=> Props have no built-in default value if missing it gives undefined.

Ex:

```
function Greet(props){
    return (<>
    <h1> Hello {props.name}</h1>
    </>
  );
}
function App(){
    return (<> < Greet /> </> )
}
export default App;
```

O/P: Hello

=> We can give default values to the props.

way1: (modern way) App.js:

```
function Greeting ({name = "Guest"}) {
  return <h1> Hello {name} </h1>;
}
function App () {
  return (
    <>
    <Greeting />
    <Greeting name = "A" />
    </>
  )
}
export default App;
```

O/P
Hello Guest
Hello A

way2: default inside function body.

App.js:

```
function Greeting (props) {
  const name = props.name || "Guest";
  return <h1> Hello {name} </h1>;
}
function App () {
  return (
    <> <Greeting /> </>
  )
}
export default App;
```

O/P
Hello Guest

State:

=> What are Hooks? Hooks are special functions that let you use state and other React features in functional components (without writing class components).

=> What is State?

- It is a built-in feature used to store and manage data inside a component.
- It is mutable, meaning it can change over time.
- When state changes, React automatically re-renders the component to update the UI.
- In functional components, state is managed using the useState hook.

e.g.,

```
const [count, setCount] = useState(0);
```

initial value is 0

current state value

Function used to update state

For UI update, variables are not used in React. Instead state is used

=> To import state we uses the Hook called useState from the React.

e.g.,

```
import {useState} from "react";
```

=> Ex: Printing something using useState.

App.js:

```
import {useState} from "react";
function App () {
  const [fruit, setFruit] = useState("Apple");
  return (
    <>
    <h1> {fruit} </h1>
    </>
  )
}
export default App;
```

current state value

initial state value

O/P
Apple

=> Ex: updating fruit name when button is clicked. App.js:

```
import {useState} from "react";
function App () {
  const [fruit, setFruit] = useState("Apple");
  const handleFruit = () => {
      setFruit("Banana");
  }
  return (
    <>
    <h1> {fruit} </h1>
    <button onClick = {handleFruit}>
    Click </button>
    </>
  )
}
export default App;
```

O/P
Apple
Click
when clicked
Banana
Click

=> Ex: Counter logic.

```
import {useState} from "react";
function App () {
  const [count, setCount]
     = useState(0);
  const updateCount = () => {
      setCount(count+1);
  }
  return (
    <>
    <h1> Count: {count} </h1>
    <button onClick = {updateCount}>
    Click </button>
    </>
  )
}
export default App;
```

O/P
Count: 0
Click
when clicked
Count: 1
Click

=> Ex: change status of text from "ON" to "off" and vice-versa.

at start show off

```
import...
function LightSwitch () {
  const [isOn, setIsOn] = usestate(false);
  const toggleLight = () => {
    setIsOn(!isOn);
  }
  return (
    <>
    <h1> Light: {isOn ? "ON" : "OFF"} </h1>
    <button onClick = {toggleLight}> Click
    </button>
    </>
  )
}
export default App;
```

change the status

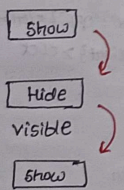
LightSwitch

O/P
Light: OFF
Click
Light: ON
Click
Light: OFF
Click

=> Ex: Show or Hide text.

```
import...
function ShowHideText(){
const[show, setShow] = useState(true);
const toggleText = ()=>{
    setShow(!show);
}
return(
<>
<button onClick={toggleText}>
{show ? "Hide": "Show"}</button>
{show && <p>visible</p>}
</>
)
}
export default showHideText;
```

O/P



doing Helps in some extra work after the UI renders

=> Library vs Framework :

Library	Framework
① We call the library. We control the app^n flow.	① It calls the code. It dictates the flow.
② Highly flexible.	② Follows rules & regulations.
③ Focus on a single, narrow functionality.	③ Offers a complete solution for building an entire app^n.
④ E.g., React, jQuery	④ E.g., Angular, Next.js

=> What is vite? It's a modern frontend tool that gets your web dev environment up & running in milliseconds?

=> Why vite for React?
- Faster development startup
- Vite updates only the changed module instead of rebuilding the entire application.
- Simpler configuration

=> Execution flow of React :
In React + Vite app, the browser first load the index.html, which then runs main.jsx. The main.jsx file starts React and renders the App.jsx component. From there, App.jsx loads and displays other component. When data or state changes, React re-renders only the affected component & updates the page without refreshing the whole website.

useEffect :

=> What is useEffect?
- It's a React hook used to run some code after the component renders.
- It's a React hook used to handle "side effects".
- It's a React hook that lets you synchronize a component with an external system.

=> What does side effects mean?
Things that happens outside the normal rendering process, e.g., Fetching data from API, timers, updating the browser, etc.

=> Syntax :

```
useEffect(()=>{
    //side effect code
},[dependencies]);
```

Part 1: Function → what to do
Part 2: dependency array (optional) → when to do it

=> While useEffect snippet looks like this :

```
useEffect(()=>{
  //first  → side effect logic
  return()=>{
  //second → cleanup function
  }  → dependency
},[third])
```

=> first - Side effect Logic
This code runs:
- after the component renders for the first time.
- again whenever any dependency in the array changes.

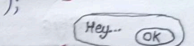
=> second - Clean up function (The fun^n returned from useEffect is the cleanup fun^n)
It runs:
- before the effect runs again due to dependency changes.
- when the component mounts.

=> third - dependency
value listed in the dependency array tell React when to re-run the effect.

=> Variation 1 : Runs on every render.
App.jsx :

```
import {useEffect} from "React";
function App(){
  useEffect(()=>{
    alert("Hey...");
  })
  return(
  <>
  <h1>Aditya</h1>
  </>
  )
}
export default App;
```

alertbox → O/P



Aditya

Here, useEffect is used without a dependency

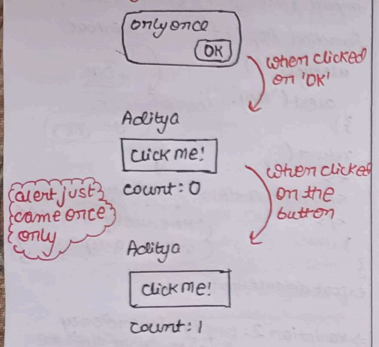
=> Variation 2 : empty dependency array. Runs only once, right after the component is rendered for the first time.
App.jsx :

```
import {useState, useEffect} from
'react';
function App(){
  const[count, setCount] = useState(0);
  useEffect(()=>{
    alert("only once");
  },[])
  function handleClick(){
     setCount(count+1);
  }
  return(
  <>
  <h1> Aditya </h1>
  <button onClick = {handleClick}>
  Click me! </button>
  <br/>
  <h2> Count: {count} </h2>
  </>
  )
}
export default App;
```

dependency array is empty

O/P

1st time loading:

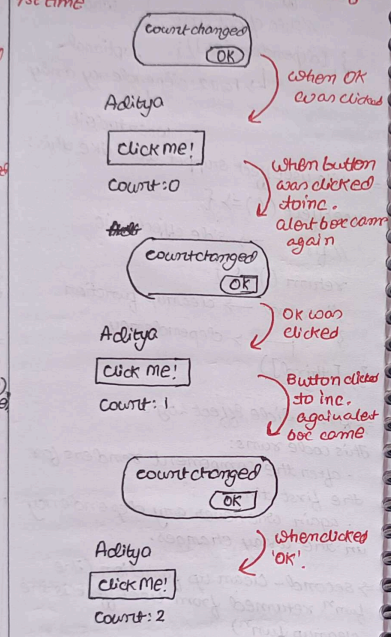


⇒ variation 3: dependency array is not empty. Runs every time (1st on render) ~~App.jsx:~~ ~~and~~ the value of array changes

```
import...
function App(){
 const [count, setCount] = useState(0);
 useEffect(()=>{
    alert("count changed");
 },[count])
  //rest from last code
 )
}
export default App;
```

O/P

when screen loads: alert box came for 1st time

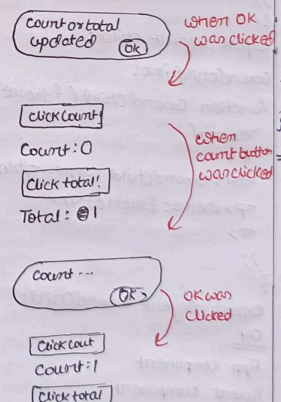


⇒ variation 4: multiple dependencies. React watch the dependencies, whenever either one is updated, the code inside useEffect executes.

App.jsx:

```
import...
function App() {
   const [count, setCount] = useState(0);
   const [total, setTotal] = useState(1);
   useEffect(()=>{
    alert("count or total updated");
   }, [count, total])
   function handleCount(){
      setCount(count+1);
   }
   function handleTotal(){
      setTotal(total+1);
   }
   return (
   <>
   <button onClick={handleCount}>
   Click Count! </button>
   <br/>
   <h2> Count: {count} </h2>
   <button onClick={handleTotal}> Click total!
</button><br/>
     <h2> Total: {total} </h2>
     </>
   )
}
export default App;
```

O/P



⇒ variation 5: with cleanup function.

For the snippet:

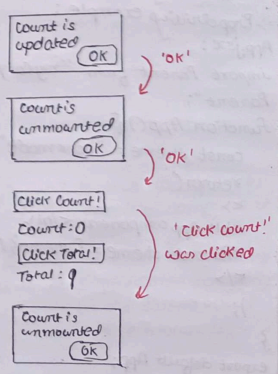
```
useEffect(()=>{
   alert("count is updated");
   return ()=>{
   alert("count is unmounted")
  }
},[count])
```

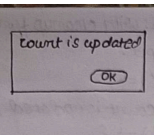
Here, whenever count changes, React first clears the previous effect, then runs the new one.

App.jsx:

```
import...
function App(){
const [count, setCount] = useState(0);
const [total, setTotal] = useState(1);
useEffect(()=>{
   alert("count is updated");
   return ()=>{
   alert("count is unmounted");
   }
},[count])
//rest from last code
 )
}
```

O/P





useContext:

=> What is Prop Drilling? Prop drilling is the procen of passing data from a parent component to a deeply nested child component through multiple levels of components using props.

=> Why prop drilling is an issue:

1. Lots of Repeated code
2. Harder to maintain
3. Intermediate Components become unnecenary menengers.

=> Prop drilling is not a hook.

=> Prop drilling example:

App.jsx:

```
import Parent from ".myComponent/Parent";
function App(){
  const theme = "dark mode";
  return(
  <>
  <h1> App component </h1>
  <Parent theme={theme}/>
  </>
  );
}
export default App;
```

Parent.jsx:

```
import Child from "./Child";
function Parent({theme}){
 return(
 <>
 <h2> Parent component </h2>
 <Child theme={theme}/>
 </>
 )
}
export default Parent;
```

Child.jsx:

```
import GrandChild from "./GrandChild";
function Child ({theme}){
return(
<>
<h3> Child component </h3>
<GrandChild theme={theme}/>
</>
);
}
export default Child;
```

GrandChild.jsx:

```
function GrandChild ({theme}){
 return.(
 <>
 <h4> GrandChild compo. </h4>
 <p> Theme: {theme} </p>
 </>
 );
}
export default GrandChild;
```

O/P

```
App component
Parent component
Child component
GrandChild comp.
Theme : Dark mode
```

=> Is Prop drilling an issue? Yes. We can use useContext.

=> What is useContext? It's a Reacthook that lets a component accen data from a context without passing that data through props at every level.

=> useContext has 3 basic steps:

1. Create Context
2. Provide
3. Consume

=> Example of useContext:

App.jsx:

```
import {createContext, usestate}
from "react";
import ChildA from "./myComp/ChildA";
const UserContext = createContext();
function App(){
  const [user, setUser] = usestate({
name: "Aditya"});
return(
<>
<UserContext.Provider value={user}>
<ChildA/> </UserContext.Provider>
</>
 )
}
export default App;
export {UserContext};
```

→ create context. This creates a global container.

→ Pass the value (here we created)

→ wrap all the child inside a provider. Basically we are wrapping every child which we want to be consumer.

→ don't forget to export UserContext

ChildA.jsx:

```
import React from "react";
import ChildB from "./ChildB";
function ChildA(){
    return(
      <> <ChildB/> </>
    )
}
export default ChildA;
```

ChildB.jsx:

```
import {UserContext} from '..App';
function ChildB(){
const user = useContext(UserContext);
return(
<div><h1>{user.name}</h1></div>
)
}
export default ChildB;
```

O/P

```
Aditya.
```

Routing:

=> What is Route? It is a specific URL path mapped to a specific component or view in your application.

=> What is Routing? It's the mechanism that allows a web application to change what the user sees on the screen based on the on the URL in the browsers address bar.

=> Why to prefer react-router-dom?
- Declarative Routing
- Nested layouts
- Performance optimization

=> Command to install react-router-dom:
npm i react-router-dom

=> To import:
```
import {createBrowserRouter,
RouterProvider} from 'react-router-dom';
```

=> Example:
App.jsx:
```
import...
const router = createBrowserRouter([
 {path: "/", element: <div><Home/>
  <Navbar/></div>},
 {path: "/about", element: <div><About/>
  <Navbar/></div>},
]);
function App(){
 return (
  <div>
  <RouterProvider router={router}/>
  </div>
 );
}
export default App;
```

This fn is used to declare all your app paths and their corresponding components

This component activates router in your app

Home.jsx and About.jsx be simple ones

Navbar.jsx:
```
import...
import {Link} from 'react-router-dom';
function Navbar(){
return (
 <>
  <ul><li><Link to='/'>Home</Link></li>
  <li><Link to='/about'>About</Link></li></ul>
 </>
)
}
export default Navbar;
```

O/P

localhost:5173
Home page
- Home
- About

When click on 'About'

localhost:5173/about
About page
- Home
- About

=> How it worked (not reloaded)?
1. React Router changes the URL
2. It checks which route matches the new URL.
3. It unmounts the old route component (e.g., <Home/>)
4. It mounts the new route component (e.g., <About/>).
5. Components that are outside the changing area (like Navbar) remain on the screen and are not recreated.

=> Dynamic routes: URL with params.

=> How to define a Route Parameter?
```
{path: "/student/:id", element: <div>
<ParamComp/> <Navbar/></div>}
```
ParamComp.jsx:
```
import...
import {useParams} from 'react-router-dom';
function ParamComp(){
  const {id} = useParams();
  return (<> Params: {id} </>)
}
export default ParamComp;
```

To access the actual value typed in the URL inside your component, React Router provides a built in hook called useParams.

=> What is useNavigate hook? This hook is used for programmatic navigation. It allows you to redirect users to different pages using code logic rather than relying on a direct user click on a <Link> component.

=> When to use <Link>? It is declarative. For static navigation where the user directly initiates the change by clicking.

=> When to use useNavigate? Programmatic. For dynamic navigation triggered by background code or after an event finishes executing.

=> Ex: button to navigate to the dashboard.
Home.jsx:
```
import...
import {useNavigate} from 'react-router-dom';
function Home(){
 const navigate = useNavigate();
 function handleClick(){
  navigate('/dashboard')
 }
 return(<>
 <button onClick={handleClick}>Todashb
 </button></>
 )
}
export default Home;
```

declaration

address

O/P

localhost:5173
Home Page [Todashb]
- Home
- About

When clicked on the button

localhost:5173/dashboard
Dashboard Page
- Home
- About

=> What is Nested Routing? It's the practice of rendering routes inside other routes. It allows a part of page (like sidebar etc) to be exactly the same while other parts to be dynamically swap content based on the sub-path.

=> To set up nested routing, we pass an array of routes to the children property of a parent route.

=> Ex: Nested Routing.
App.jsx:
```
import...
import {createBrowserRouter,
RouterProvider} from 'react-router-dom';
const router = createBrowserRouter([
{path: "/", element: <div><Navbar/>
<Home/></div>},
{path: "/dashboard", element: <div><Navbar
<Dashboard/></div>, children: [{index:
true, element: <div>Overview Content
</div>}, {path: "settings", element: <div>
Settings Content</div>}]},
]);
function App(){
  return <div><RouterProvider router=
{router}/></div>;}
```

Dashboard.jsx:

```
import ...
import {Outlet} from 'react-router-dom';
function Dashboard(){
  return (
    <>
    Dashboard page <hr/>
    <Outlet/> </>
  )
}
```

O/P

localhost:5173/dashboard
- Home
- About

Dashboard Page

Overview content

localhost:5173/dashboard/settings
- Home
- About

Dashboard Page

Settings Content

useRef Hook:

=> This hook is used to store a value that persists b/w renders without causing a re-render, when it changes.

=> When to use useRef?
1. Accessing DOM elements directly
2. Storing mutable values.
3. Preserving values b/w renders without triggering updates.

=> Syntax:

```
import {useRef} from "react";
const myRef = useRef(initialValue);
```

=> Ex: to make input field get focused on button click.

```
import ...
function App(){
  const inpRef = useRef(null);
  const inpHandler = ()=>{
  inpRef.current.focus();
  inpRef.current.style.color="red";
}
return (
  <>
  <h1> Adi </h1>
  <input ref={inpRef} type="text"
placeholder="Your name"/>
  <button onClick={inpHandler}>
  Click </button>
  </>
)
}
export default App;
```

O/P

Adi

Your Name | Click

When clicked

Focused input field with red text

Adi

Aditya | Click

=> use case of useRef:
1. when a variable persists its value across every re-render.
2. when we want to access a DOM element directly.

=> Ex: When variable can't manage to persists its value across re-renders.

App.jsx:

```
import...
function App(){
  const [count, setCount] = useState(0);
  let val = 1;
  function handleInc(){
    val = val+1;
    console.log("val is:", val);
    setCount(count+1);
  }
  return(
  <>
  <button onClick={handleInc}>
  Increase </button>
  <br/>
  Count: {count}
  </>
  )
}
```

simple variable is used to store val variable

Code will not work as intended

O/P

Increase
Count: 0

Console:

when clicked 9 times

Console:
val is: 2

Increase
Count: 9

Since, val is a normal variable, every time the component re-renders (bez of button click), it resets back to 1.

=> Ex: solution of last issue, as well as example of use case 1 of useRef.

App.jsx:

```
import...
function App(){
  const [count, setCount] = useState(0);
  let val = useRef(0);
  function handleIn(){
    val.current = val.current+1;
    console.log("val is:", val);
    setCount(count+1);
  }
  //code
}
```

O/P

Increase
Count: 0

Console:

Increase
Count: 4

Console:
val is: 1
val is: 2
val is: 3
val is: 4

=> Normal variable:
- doesn't persists after re-render
- don't causes re-renders
- can reset on render

=> useRef:
- Persists after re-renders
- don't causes re-renders
- can't resets on re-renders.

Uncontrolled component:

=> Key characteristics:
- the DOM stores the form data
- React doesn't control the value in real time.
- We typically use useRef to access the value

~~Ex~~

=> Controlled vs uncontrolled compo:

Feature	Controlled Compo	Uncontrolled
State location	React state	DOM
updates	onChange	No React updates on typing
Data accen	Always in State	Accen via ref

=> Ex: form controlled by DOM. In this example we are printing value entered in the console. (without using useRef)

App.jsx:

```
import...
function App(){
    const handleForm = (event)=>{
    event.preventDefault();
    const user = document.querySelector
    ("#user").value;
    const password = document.querySelector
    ("#password").value;
    console.log(user,password);
  }
  return(
  <>
  <form action="" method="post"
onSubmit={handleForm}>
    <input type='text' id='user'
    placeholder='Name' /><br/><br/>
    <input type='password'
    id='password' placeholder='password'/>
    <br/><br/>
    <button> Submit </button>
  </form>
  </>
 )
}
export default App;
```

O/P

Console

Name

Password

Submit

typed by us

Console

Aditya 1234

Aditya

1234

Submit

Since, React doesn't stored the input values in state, it's an uncontrolled component.

=> Ex: Handling the above using useRef

App.jsx:

```
import...
function App(){
   const userRef = useRef();
   const passRef = useRef();
   const handleForm = (event)=> {
   event.preventDefault();
   const user = userRef.current.value;
   const pass = passRef.current.value;
   console.log(user,pass)}
};
return(
  <>
  <form onSubmit={handleForm} >
  <input type="text" ref={userRef}
   placeholder="Name"/>
  <br/><br/>
  <input type="password" ref={passRef}
  placeholder="Password"/>
  <br/><br/>
  <button> Submit </button>
  </form></>
 );
}
```

O/P

Console

Name

Password

Submit

typed by us

console

Aditya 1234

Aditya

1234

Submit

Passing Props as Children and Functions:

=> What is children? It's a special prop in React that contains everything you put between a component's opening and closing tags.

e.g.,

```
<Card name="Aditya">
  <h1>Yes!</h1>
  <p>Hey</p>
</Card>
```

(h1 and p → children)

=> props.children is a special React prop that represents the content placed inside a component.

=> Ex: basic example of props.children.

App.jsx:

```
import...
function App(){
  return(
  <>
  <Card name="Aditya">
     <h1>Hey!</h1>
     <p>Yes!</p>
  </Card>
  </>
  )
}
```

card.jsx:

```
import...
function Card(props){
  return(
  <>
  {props.children}
  </>
  );
}
```

O/P

Hey!

Yes!

=> Another way to create children is using the "children=" :

```
<Card children="Aditya">
<h1>Yes!</h1>
</Card>
<Card children="Utsav">
</Card>
```

& we do:

```
<>
{props.children}
</>
```

O/P

Yes!

Utsav

=> Ex: passing a function from parent to child.

App.jsx:

```
import...
function App(){
  const [count, setCount]=useState(0);
  function handleClick(){
    setCount(count=> count+1);
  }
  return(
   <>
   <h1> {count} </h1>
   <Button
     incCount={handleClick}
     text="Click me"/>
   </>
  );
}
```

Button.jsx:

```
import...
function Button(props){
  return(
   <>
   {props.children}
   <button onClick={props.incCount}>
   {props.text}
   </button>
   </>
  );
}
```

O/P

0 1

Click me Click me

forwardRef:

=> What is forwardRef? It is a method (React API) that allows a parent component to pass its ref through a custom child component and access the child's underlying DOM element or component instance.

=> Ex: input field get focused with value '100' when clicked on button.

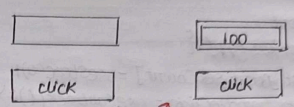
UserInput.jsx:

```
import React, {forwardRef} from 'react';
const UserInput = forwardRef((props,ref)
=> {
 return(
  <>
  <input type='text' ref={ref}/>
  </>
 );
});
export default UserInput;
```

App.jsx:

```
import...
function App(){
  const inputRef =useRef(null);
  const updateInput =()=>{
    if(input.current){
     input.current.value=100;
     input.current.focus();
    }
  };
  return(
   <>
   <UserInput ref={inputRef}/>
   <button onClick={updateInput}>
   Click </button>
   </>
  );
}
```

O/P



useMemo:

=> It remembers the result of a calculation and recalculates it only when its dependencies changes.

=> It is used to memoize (cache) the result of an expensive calculation so that React doesn't recompute it on every render.

=> Syntax:

```
const memoizedValue = useMemo
(()=> {
  return someExpensive();
}, [dependencies]);
```

=> What is an expensive calculation?

```
function expenTask(num){
   for(let i=0; i<=1000000000;i++){}
   return num*2;
}
```

Here, to multiply 'num' with 2 it has to go through loop, which is quite big.

=> Ex: when expensive calculation actually affects the code. (without using useMemo)

App.jsx:

```
function App(){
  const [count, setCount]=useState(0);
  function expTask(num){
    for(let i=0; i<=1000000000;i++){}
    return num*2;
  }
  let doubleValue = expTask(4);
  return(
   <>
   <button onClick={()=> setCount
   (count+1)}>
   Increement </button>
   <div>
     count: {count} </div>
   <div> Double: {doubleValue}
   </div>
   </>
  )
}
```

O/P

Value of count can take some extra time now to increase.

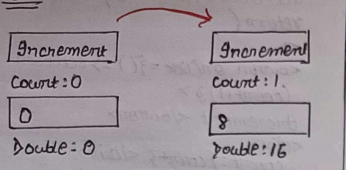
Increement	Increement
Count: 0	Count: 1
Double: 8	Double: 8

=> Ex: Above example issue can be solved using useMemo hook.

App.jsx:

```
import...
function App(){
  const [count, setCount]=useState(0);
  const [input, setInput]=useState(0);
  function expTask(num){
    //same code
  }
  const doubleValue = useMemo(()=>
  expTask(input),[input]);
  return(
   <><button onClick={()=> setCount(count
   +1)}> Increement </button>
   <div> Count: {count} </div>
   <input type="number" value={input}
  onChange={(e)=> setInput(Number(e.
  target.value))} />
   <div> Double: {doubleValue} </div>
  </>
  );
}
```

O/P



Now, count will increase quickly, while the double calculation will take time for sure

useCallback :

=> It's a built in React hook that lets you cache (memoize) a function definition between re-renders of a component.

=> useMemo
↳ A memoized value

useCallback
↳ A memoized function

=> We know, React.memo can also prevent re-rendering until a value is passed. But if we pass a function, then it will not prevent re-rendering.

=> But, why if fun is involved then re-rendering is not prevented? Every time App re-renders, the fun is recreated, since it is recreated thus for every creation there is a new reference, bez the fun reference change on every render. So, React.memo re-renders.

=> Ex: basic example. App.jsx:

```
import...
Function App(){
 const [count, setCount] = useState(0);
 const handleClick = useCallback(()=>{
    setCount(count+1);
 },[count]);
 return(
  <>
  <h1> Count: {count} </h1> <br/> <br/>
  <button onClick={handleClick}>
  Increment </button> <br/> <br/>
  <ChildComponent buttonName="click"
  handleClick={handleClick} />
  </>
 )
}
```

Child Component.jsx:

```
import...
Function ChildComponent(props){
 console.log("Child is re-rendered");
 return(
  <>
  <button onClick={props.handleClick}>
  {props.buttonName}
  </button>
  </>
 )
}
```

O/P

Count: 4

Increment

Click

console

Child comp is re-rend.

(4) " " "

Updating Objects in State :

=> State should be treated as immutable. That means when you want to update an object in state, you create a new object rather than modifying the existing one.

=> Ex: basic update in object and displayed change on UI.

App.jsx:

```
import...
function App(){
 const [data, setData] = useState({
    name: 'Aditya',
    address: {
       city: 'Mug',
       country: 'Sup'
    }
 })
 const handleName = (val)=>{
    let tempData = data;
    tempData.name = val;
    setData({...tempData})
 }
 return (
  <>
  <input type='text' placeholder='Name'
  onChange={(event)=>handleName
  (event.target.value)}/>
  <h2>Name: {data.name}</h2>
  <h2>City: {data.address.city}</h2>
  </>
 )
}
```

main logic

O/P

we typed

utsav

Name: utsav

City: Mug

Updating Arrays in State :

=> Array...
//Same as this
↳

=> Ex: basic update the last element of array and showing on UI.

App.jsx:

```
import...
function App(){
const [data, setData] = useState([
 'adi', 'uts', 'pihu'
])
const handleUser = (name)=>{
    data[data.length-1] = name;
    setData([...data])
}
return(
 <>
 <input type='text' placeholder='Last name'
 onChange={(e)=>handleUser(e.target.
 value)}/>
 {
 data.map((item,index)=>{
  <h3 key={index}>{item}</h3>
 ))
}
```

O/P

we typed

pihu

adi
uts
pihu

React Hook Form :

=> It's a library for managing forms in React applications using React Hooks.

=> It helps handle :
- Form state management
- Input registration
- validation
- Error Handling
- Form submission

while minimizing unnecessary re-renders, making forms faster.

=> Command to install it :

npm install react-hook-form

=> Ex : keeping 'firstname' as required field, min length of it to be 3, showing error menage, and logging the data on console when submitted.

```
import...
function App(){
  const {
        register, handleSubmit,
        watch, formState: {errors},
      } = useForm()
  function onSubmit(data) {
     console.log(data);
  }
  return(
  <>
  <form onSubmit={handleSubmit(onSubmit)}>
  <div> <label> F. Name : </label>
  <input {...register('FirstName',
     { required: true,
       minLength: {value: 3, menage: "Min
       atleast 3"} })}/>
```

useForm() gives me the tools (register, etc) that I need to manage the form.

register() connects an input field to React hook so it can track.

to make it mandatory

```
{errors.FirstName &&
  <p> {errors.firstName.menage}</p>
</div><br/>
<div><label>
Last Name: </label>
<input {...register('lastname')}/>
</div>
<input type='submit'/>
</form>
)
}
```

check if F.Name field has validation error, and if does, display the msg.

written by us

O/P

F.Name : Adi

Min Len atleast 3

Last Name :

Submit

Console

F.Name : Adi

L.Name : Kr

Submit

{'Adi', 'Kr'}

useActionState Hook :

=> It's a React Hook that helps manage the state resulting form a form action.

=> It can be used to : submit form, validation results, etc.

=> Take user input, display on console, have a waiting period too, also 'Submit' button shall also get disabled when clicked for submitting.

App.jsx :

```
import...
function App(){
  const handleSubmit = async(
previousData, formData) => {
  let name = formData.get('name');
  let password = formData.get('password');
  await new Promise(res => setTimeout
  (res, 2000));
  console.log("handleSubmit called",
  name, password);
}
const [data, action, pending] =
useActionState(handleSubmit, undefined);
return(
<>
<form action={action}>
<input type='text' placeholder='Enter
your name' name='name'/>
<br/><br/>
<input type='password' placeholder=
'Enter password' name='password'/>
<br/><br/>
<button disabled={pending} >
 Submit </button></form> </>
)
}
```

handleSubmit is the action fn^n that runs when the form is submitted.

promise added to set delay b/w submission & console.

state

submitHandlerForForm

loading status

to disable button when form is not submitted

O/P:

Aditya

.....

Submit

console:

handleSubmit called Aditya 1234

useId Hook :

=> It is a React hook that generates a unique and stable ID for a component.

=> Syntax:

```
const id = useId();
```

=> Ex: show id generated are unique.

App.jsx:

```
import...
function App(){
  const name = useId();
  const terms = useId();
  const cond = useId();
  return(
  <>
  <h1>{name}</h1>
  <h1>{terms}</h1>
  <h1>{cond}</h1>
  </>
  )
}
```

O/P

```
_r_0_
_r_1_
_r_2_
```

Custom Hooks :

=> These are reusable JS functions that let you share stateful logic b/w React components.

=> A custom hook:

- Starts with the word 'use'.
- Can use other React hooks (useState, etc)
- Can call other custom hooks
- Returns values, functions, or objects that components can use.

=> Syntax:

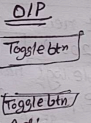
```
function useCustomHook() {
    //Hook logic
    return value;
}
```

=> Ex: useToggle hook created by us to show & hide text.

App.jsx:

```
import...
function App(){
  const [value, toggleValue] = useToggle
  (true);
  return(
  <>
  <button onClick={toggleValue}>
  Toggle btn </button>
  {
    value ? <h1>Adi</h1>
  }
  </>
  )
}
```

O/P



useToggle.jsx:

```
import...
const useToggle(defaultVal) => {
  const [value, setValue] = useState();
  function toggleValue(val){
    if(type of val != 'boolean'){
      setValue(!value);
    } else{
      setValue(val);
    }
  }
  return [value, toggleValue];
```

API :

=> What is an API? An API (App^n Programming Interface) is usually a way for your React app^n to communicate with the server or another app^n to get or send data.

=> Common API methods:

- GET
- POST
- PUT/PATCH
- DELETE

=> Ways to call APIs in React:

- fetch()
- Axios

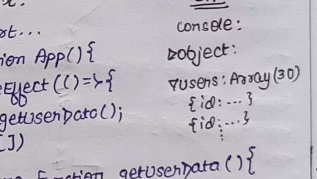
=> Printing the API data to console:

App.jsx:

```
import...
function App(){
  useEffect(()=>{
    getUserData();
  },[])

  async function getUserData(){
    const url = "https://abc.com/users";
    let response = await fetch(url);
    response = await response.json();
    console.log(response);
  }
  return(
  <><h1>Aditya</h1></>
  )
}
export default App;
```

O/P



=> Ex: Renden the firstname and secondname on screen.

App.jsx:

```
import...
function App(){
  const [usersData, setUsersData]
  =useState([]);
  useEffect(()=>{
    getUserData();
  },[])
  async function getUserData(){
    const url = "https://ABC.com/users";
    let response = await fetch(url);
    response = await response.json();
    setUsersData(response.users);
  }
  return(
  <>
  <h1>Aditya</h1>
  {usersData.map((user)=>{
    <ul key={user.id}>
      <li>{user.firstName}</li>
      <li>{user.lastName}</li>
    </ul>
  })} </>
  )
}
```

O/P

Aditya

- Emily
- Johnson
- Michael
- williams
- ⋮

JSON Server :

=> What is JSON? JavaScript Object Notation is a lightweight format used to store & exchange data b/w applications.

=> Why JSON? It is commonly used in APIs because it is easy for both humans and programs to read.

=> What is JSON server? It is a tool that creates a fake REST API from a JSON file.

=> To install JSON server:

```
npm install json-server
```

=> To run the JSON server:

```
npx json-server db.json
```

=> To use the APIs and reflect it on screen, use the way we did for API in last section.

=> To POST:

```
const addPost = async () => {
  const url = "http://localhost:3000/posts";
  const response = await fetch(url, {
    method: "POST", (or "PUT", or "PATCH")
    headers: {
      "content-Type": "application/json",
    }
    body: JSON.stringify({
      id,
      title,
      views: Number(views),
    }),
});
```

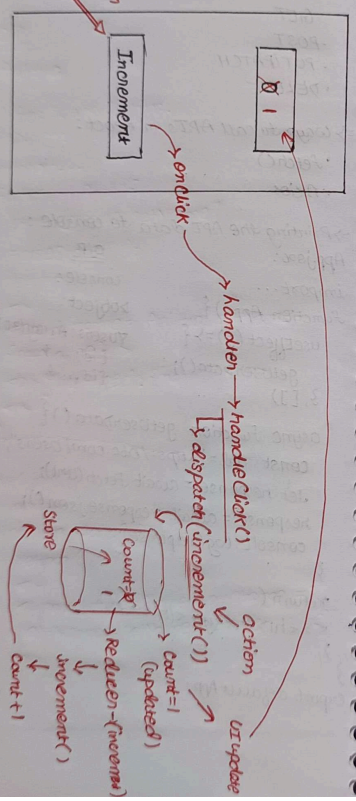
Redux:

=> It's a state management library often used with React to handle and centralize application state in a predictable way.

=> Core concepts of Redux:

1. Action
2. Reducer
3. Slice
4. Store
5. State

=> Flow:



=> Basically:

Component → Action → Reducer/Slice → Store → State → Component updates

What happened?

How should state change?

Central place managing all State & reducers

Feature specific Redux logic (action + reducers in RTK)

Current data

=> To install redux toolkit:

```
npm install @reduxjs/toolkit
npm install react-redux
```

=> Ex: Counter.

App.jsx:

```
import...
function App(){
  const count = useSelector((state)
  => state.counter.value)
  const dispatch = useDispatch();
  function handleIncClick(){
      dispatch(increment());
  }
  function handleDecClick(){
      dispatch(decrement());
  }
  return(
  <>
  <button onClick={handleIncClick}>
  +
  </button> <br/> <p> Count: {count} </p>
  <button onClick={handleDecClick}>
  -
  </button>
  </>
  )
}
```

counterSlice.jsx:

```
import...
export const counterSlice = createSlice({
  name: 'counter',
  initialState: {
    value:0,
  },
  reducers: {
    increment: state => {
      state.value +=1
    },
    decrement: state => {
      state.value -=1
    },
    incrementByAmount:(state, action)=>
    {
      state.value += action.payload
    }
  }
})
export const {increment, decrement,
incrementByAmount} = counterSlice.actions
export default counterSlice.reducer
```

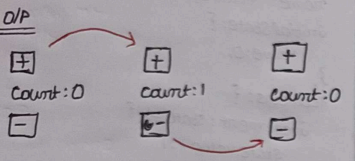
store.js:

```
import...
export const store = configureStore({
    reducer:{
      counter: counterReducer
    },
})
```

Main.jsx :

```
import...
createRoot(document.getElementById
('root')).render(
  <Provider store={store}>
  <App/>
  </Provider>
)
```

O/P



State Lifting :

=> It's the procen of moving state from a child compement to its nearest common parent so that multiple components can share and stay synchronized with the same data.

=> It creates a single source of truth and allows data to be paned down through props.

=> Ex : No state liftiup (passing data from parent to child)

App.jsx :

```
import...
function App() {
  return (
    <>
    <Card name="Aditya" />
    </>
  )
}
```

Card.jsx :

```
import...
function Card(props) {
  return (
    <>
    {props.name}
    </>
  )
}
```

O/P

Aditya

=> Ex : state lifting.

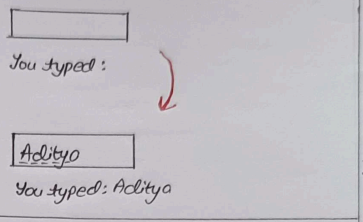
App.jsx :

```
import...
function App() {
  const [name, setName]
  = useState('');
  return (
  <>
  <Card name="Aditya" setName=
  {setName}/>
  <p>You typed : {name} </p>
  </>
  )
}
```

Card.jsx :

```
import...
function Card(props) {
  return (
  <>
  <input type="type" onChange={
  (e) => props.setName(e.target.value)}/>
  </>
  )
}
```

O/P



Zustand :

=> It's a small, lightweight state management library for React. Its pupose is to let multiple components share state without prop drilling or excenive context providers.

=> To install it :

```
npm install zustand
```

=> Ex : courseStore.js :

```
import...
const courseStore = (set) => ({
  courses : [],
  addCourse : (course) =>
    set((state) => ({
      courses : [course, ...state.course],
    })),
  removeCourse : (courseId) =>
   set((state) => ({
   courses : state.courses.filter(
   (course) => course.id !== courseId
   ),
   })),
  toggleCourseStatus : (courseId) =>
   set((state) => ({
    courses : state.course.map((course)
    =>
    course.id === courseId
    ? {
      ...course,
      completed : !course.completed,
      }
    : course
  ),
  })),
});
const useCourseStore = create(
  devtools(
    persist(courseStore, {
      name : "courses",
    })
  )
);
```

courseForm.jsx :

```
import...
const CourseForm = () => {
  const addCourse = useCourseStore
  ((state) => state.addCourse);
  const [courseTitle, setCourseTitle]
  = useState("");
  console.log("CF submitted");
  const handleCourseSubmit = () => {
  if (!courseTitle) {
    alert("Please add a CT");
    return;
  }
  addCourse({
  id : Math.ceil(Math.random()* 10000),
  title : courseTitle,
  completed : false,
  });
  setCourseTitle("");
};
return (
  <>
```

```
<>
<h2> Add Course </h2>
<input type="text" value={
courseTitle} onChange={(e) =>
setCourseTitle(e.target.value)}/>
<button onClick={handleCourse
Submit}>
Add Course
</button>
</>
);
};
```

O/P

Add Course

	Add Course

courses

- ReactJs Not completed.